ARTELIER

Designing dashboards for data-rich applications

Twelve principles for analytics, admin panels, business intelligence and internal tools — the screens people work in rather than look at.

A DASHBOARD AND DATA-UI STUDIO

artelier.agency

October 2026

Preface

Most dashboards are not wrong. They are unreadable. The figures reconcile, the pipeline is sound, the charts are drawn correctly — and still nobody can say what to do on Monday morning. The failure is almost never in the data layer. It is in the decision the screen declined to make on the reader's behalf.

These twelve principles are what we work from. They are deliberately narrow: they apply to data-rich interfaces — analytics, admin panels, business intelligence, observability, internal tools — where a reader arrives with a question and leaves with or without an answer. They are not general visual design advice, and several of them would be poor guidance for a marketing site.

Each principle states a rule, explains the reasoning, and gives the specific moves it implies. Where we have been wrong about one, it is noted. A principle that has never cost anything to follow is usually not a principle.

The principles

01 Rank, don't list — A screen that shows everything has decided nothing.

02 Design for the question, not the data — Start from the decision the screen exists to support.

03 Spend the accent once — Colour that decorates makes colour that means invisible.

04 Set numbers in a monospaced face — Columns are read downward; proportional digits break the column.

05 Cards, not strips — Alignment should come from one shared gutter, not from rules between bars.

06 Make density a setting — The analyst and the executive are not reading the same screen.

07 Design the empty state first — It is the first screen every user sees, and usually the last one designed.

08 A number without a threshold is trivia — 4.2% is not information until the screen says what good looks like.

09 Show the shape before the value — Position is read faster than digits, and trend is usually the question.

10 State belongs in the URL — A view that cannot be sent to a colleague will be rebuilt by hand.

11 Latency is a design material — Decide what the screen says during the two seconds it has nothing to say.

12 Accessibility is structural, not cosmetic — It cannot be added in the last sprint, because it is mostly architecture.

01

Rank, don't list

A screen that shows everything has decided nothing.

Ten thousand rows sorted by severity is not a to-do list; it is the same problem in a new order. Severity alone says nothing about how much work a fix represents, how many readers it touches, or whether two hundred findings are one template change or two hundred separate jobs.

Ranking is the expensive part of the work and the part most teams skip, because it requires a position: you must decide what matters and be accountable for the order. A product that ranks will occasionally be wrong in public. A product that lists is wrong privately, every day, by making every reader do the ranking alone.

In practice

— Score each item on impact, reach and effort, then sort by the score rather than by severity.

— Show the reasoning behind the rank on demand — a ranking nobody can interrogate will not be trusted twice.

— Collapse findings that share a single fix into one job, and say how many underlying instances it covers.

02

Design for the question, not the data

Start from the decision the screen exists to support.

The usual sequence is backwards: the warehouse has these tables, so the dashboard has these charts. The result is an interface shaped like a schema, which is a shape no human question has.

Begin instead by writing down, in a sentence each, the decisions this screen is meant to support and who makes them. Most screens support one or two. Everything that does not serve them is a second screen, a drill-down, or an export — and a surprising amount of it is nothing at all.

In practice

— Write the three questions the screen answers at the top of the design file, and delete any element that serves none of them.

— Name charts after the question, not the metric: "Are we losing new users in week one?" rather than "Weekly retention".

— If a stakeholder cannot say what they would do differently given the number, the number is reporting, not a dashboard.

03

Spend the accent once

Colour that decorates makes colour that means invisible.

In a data interface, colour is a channel carrying information: severity, status, series identity, threshold breach. Every element that borrows an accent for decoration reduces the signal of the elements using it for meaning.

The discipline is to pick one accent and reserve it for the single thing you want acted on, letting hierarchy come from weight, size and elevation instead. A monochrome interface with one accent reads faster than a six-colour one, and it degrades gracefully for the eight per cent of men with a colour vision deficiency.

In practice

— Reserve the accent for the primary action and for threshold breaches; everything else earns attention through typography.

— Never encode a state in hue alone — pair it with a glyph, a label or a position.

— Test the screen desaturated. If it stops working, colour was doing structural work it should not have been trusted with.

04

Set numbers in a monospaced face

Columns are read downward; proportional digits break the column.

A table of figures in a proportional typeface cannot be scanned vertically, because the digits do not line up and the eye has to read each value rather than compare shapes. The cost is invisible in a design review with eight rows and severe in production with eight hundred.

Monospaced — or at minimum tabular — figures turn a column into a single visual object where outliers announce themselves by width. The same applies to status codes, durations, identifiers and any value a reader will compare against its neighbours.

In practice

— Use `font-variant-numeric: tabular-nums` at minimum, a monospaced face for dense grids.

— Right-align numbers, left-align text, and never centre either.

— Keep units out of the figure and in the column header, so the digits stay in register.

05

Cards, not strips

Alignment should come from one shared gutter, not from rules between bars.

Full-width horizontal bands are the default layout of most admin interfaces, and they produce screens where nothing lines up with anything except the window. Every band sets its own padding, and the reader's eye has no vertical structure to follow.

Grouping content into cards on a common grid, with one gutter value used everywhere, gives the layout a skeleton. It also makes responsive behaviour tractable: cards reflow, bands can only squash.

In practice

— Define one gutter token and use it for the page margin, the gap between cards, and the padding inside them.

— Give every card the same corner radius; a second radius should mean a second kind of thing.

— Resist the full-bleed table. A table inside a card can have a header that stays put.

06

Make density a setting

The analyst and the executive are not reading the same screen.

Arguments about row height are really arguments about two different users. Someone reviewing a list once a quarter wants air; someone living in it for six hours wants to see forty rows without scrolling. Both are right, and picking one loses the other.

Density is cheap to make adjustable if it is a token from the start and expensive to retrofit once row heights are hard-coded in twenty components. Treat it as a first-class axis of the design system, alongside light and dark.

In practice

— Express row height, gutter and font scale as tokens that a density mode can swap.

— Remember the choice per user, not per device.

— Design the compact mode seriously rather than scaling the comfortable one down — some elements should disappear, not shrink.

07

Design the empty state first

It is the first screen every user sees, and usually the last one designed.

A dashboard with no data is not an edge case; it is day one for every customer you have not yet onboarded. "No data" in grey 12px is a dead end at exactly the moment a new user is deciding whether this product is worth the effort.

The empty state should teach: what will appear here, why it is empty, and the single next action that fixes it. The same applies to the empty state caused by a filter, which is a different problem — there, the answer is usually to show what was filtered out.

In practice

— Distinguish never-had-data from no-results-for-this-filter; they need different copy and different actions.

— Put the action in the empty state itself, not in a toolbar the reader has not looked at.

— Consider sample data for first run, clearly marked, so the reader can see the shape of the answer before they have one.

08

A number without a threshold is trivia

4.2% is not information until the screen says what good looks like.

Readers cannot evaluate a figure they have no baseline for. Shown alone, a conversion rate, an error budget or a p95 latency produces the same reaction in almost everyone: is that good? Any screen that prompts that question has transferred its work to the reader.

The fix is not a sparkline bolted on afterwards. It is deciding, with the people who own the metric, what the thresholds are — and then letting the interface state plainly which side of them you are on.

In practice

— Pair every headline figure with a comparison: target, previous period, or peer cohort.

— Encode the threshold in the visualisation — a band, a line, a change of weight — not only in a tooltip.

— Where no threshold exists, say so rather than implying one with colour.

09

Show the shape before the value

Position is read faster than digits, and trend is usually the question.

The eye reads the position of a needle, the slope of a line or the length of a bar far faster than it parses a number. For any metric that changes, the shape of the change is normally what the reader came for; the exact value is a follow-up.

This is an argument for small, dense, well-drawn graphics beside figures rather than instead of them — a sparkline next to the number, a dial where a rate is changing — not for replacing tables with pictures.

In practice

— Put a sparkline beside every headline figure that has a history.

— Prefer position and length encodings; reserve area and angle for cases where part-to-whole genuinely matters.

— Draw charts to match the design system rather than accepting a library's defaults — and if the library resists, the library is the wrong one.

10

State belongs in the URL

A view that cannot be sent to a colleague will be rebuilt by hand.

Filters, date ranges, sort order, selected rows and open panels are the reader's work. Holding them only in component state means that work cannot be shared, bookmarked, reopened after a refresh, or cited in an incident review. People respond by taking screenshots, which is a diagnosis of the interface.

Serialising view state into the URL costs an afternoon early and is a substantial refactor later. It also makes the back button behave the way the reader already expects it to.

In practice

— Encode filters and sort in query parameters; restore them exactly on load.

— Make "copy link to this view" an explicit affordance rather than relying on the address bar.

— Keep the parameters legible. A base64 blob is shareable but not editable, and people do edit them.

11

Latency is a design material

Decide what the screen says during the two seconds it has nothing to say.

Data-rich screens are slow in ways marketing pages are not: large aggregations, cold caches, third-party APIs. The loading state is therefore not an edge case but a state the reader occupies several times a session, and a spinner over the whole page throws away the layout they had already begun reading.

Treat time as something to be designed. Keep the structure on screen, show what is already known, and be honest about what is still arriving. Partial answers beat a blocked screen, provided the interface never lets a partial answer look like a complete one.

In practice

— Hold the layout with skeletons that match the real content's dimensions, so nothing shifts when data lands.

— Stream the cheap panels first rather than waiting on the slowest query.

— Mark stale or partial data explicitly, with the time it was last refreshed.

12

Accessibility is structural, not cosmetic

It cannot be added in the last sprint, because it is mostly architecture.

Contrast is the easy part. The hard parts of an accessible data interface are structural: a table that is a real table, a keyboard path through a grid, a chart with a text equivalent, focus that lands somewhere sensible when a panel opens, and live regions that announce a filtered result without announcing every keystroke.

None of these can be retrofitted cheaply, because they constrain how components are built. They are also where fashionable visual styles cause real harm — soft, borderless surfaces look superb and can leave controls with no discernible edge. A style that fails here should be bent until it stops failing, not defended.

In practice

— Make every control reachable and operable by keyboard, with a focus ring visible against its own surface.

— Give charts an accessible summary or a table equivalent; a canvas element is invisible to a screen reader.

— Respect `prefers-reduced-motion` and confirm the screen is fully usable with every animation disabled.

A note on these

These are working rules, not laws of nature. The one we revisit most is the third: a single accent is right for dense operational screens and can be too austere for a product whose users visit twice a month. The one we have never regretted is the first.

If a screen in your product fails several of these at once, that is normal — most do, including several we have shipped. The useful question is which failure is costing the most, and that is usually answerable in an afternoon of watching someone use it.

Colophon

Written by Artelier, a dashboard and data-UI studio. We design analytics interfaces, admin panels and design systems, and ship them as working front-end code. Set in Helvetica with Courier for figures. artelier.agency